Anexos — Parte 1: Interfaz de la API y Prueba de Predicción Válida (Código 200 OK)

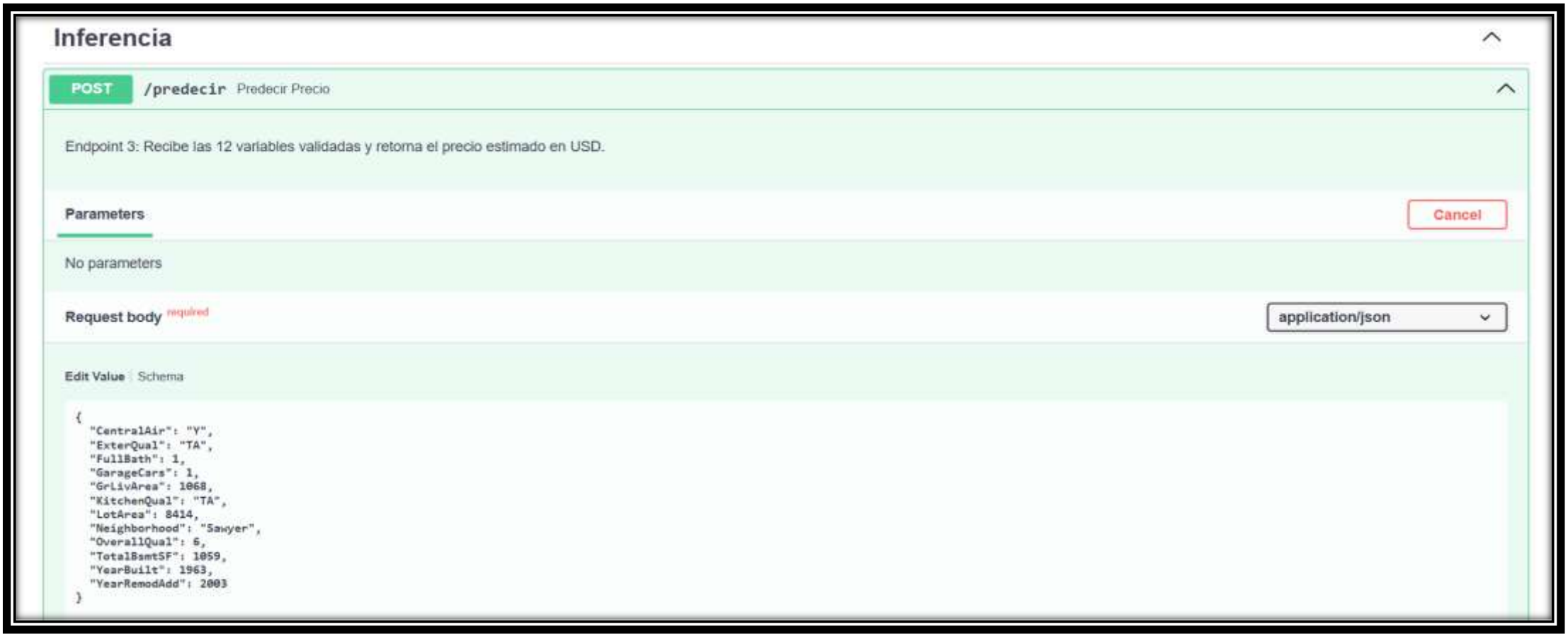
Anexo 1. Portal de Documentación Interactiva (Swagger UI) y Arquitectura de Endpoints de la API



Interpretación:

La imagen muestra la interfaz principal del servicio web en producción (**API de Predicción de Precios de Viviendas v1.0.0** bajo el estándar OpenAPI 3.1), el cual opera conectado al Pipeline de GradientBoostingRegressor entrenado con las 12 variables seleccionadas del conjunto Ames Housing. Se observa la estructura modular de los endpoints de consulta rápida: el módulo **General (GET /)**, diseñado para verificar la disponibilidad inmediata del servicio, y el módulo **Salud y Metadatos (GET /info)**, encargado de exponer las métricas de evaluación, rangos permitidos y versiones del entorno cargadas directamente desde el archivo serializado .joblib.

Anexo 2. Envío de Solicitud con Datos Válidos en el Endpoint de Inferencia (POST /predecir)



Interpretación:

Se presenta la prueba funcional de ingreso de datos sobre el endpoint **POST /predecir**, encargado de recibir las características de una vivienda en formato JSON (application/json) y validarlas a través del contrato definido en esquemas.py. En el cuerpo de la petición (Request body) se envían los 12 atributos requeridos dentro de sus rangos y categorías oficiales permitidas —incluyendo una calidad general de construcción nivel 6 (OverallQual), un área habitable de 1,068 pies cuadrados (GrLivArea), lote de 8,414 pies cuadrados (LotArea), aire acondicionado central activo (CentralAir: "Y") y ubicación en el vecindario "Sawyer"— cumpliendo íntegramente las reglas de negocio previas a la predicción.

Anexo 3. Respuesta Exitosa del Servidor (HTTP 200 OK) y Estimación del Precio en Tiempo Real



Interpretación:

Se evidencia la respuesta exitosa del servidor **uvicorn** con código de estado **HTTP 200**, confirmando que el registro superó satisfactoriamente todas las validaciones de Pydantic y fue procesado por el modelo en producción. El cuerpo de respuesta (Response body) entrega un valor estimado de **$139,223.61 USD** (prediccion_usd), cifra que coincide de manera exacta con la prueba de control realizada en el notebook de entrenamiento. Adicionalmente, la respuesta adjunta la trazabilidad completa de la predicción: el algoritmo utilizado (GradientBoostingRegressor), la precisión de referencia en datos no vistos (r2_test_referencia: 0.9162) y la marca de tiempo exacta de entrenamiento en formato UTC (2026-09-28T01:44:53...).

Anexos — Parte 2: Prueba de Rechazo y Control de Calidad de Datos (Código 422)

Anexo 4. Envío de Solicitud con Datos Inválidos para Prueba de Robustez en POST /predecir



Interpretación:

Se presenta la prueba de estrés sobre el contrato de datos (esquemas.py) en el endpoint **POST /predecir**, introduciendo deliberadamente dos anomalías de negocio dentro del cuerpo JSON (Request body). En primer lugar, se altera la variable categórica de aire acondicionado central ingresando un valor inexistente ("CentralAir": "TAL_VEZ", cuando el dominio solo admite "Y" o "N"). En segundo lugar, se asigna a la calidad general de construcción un valor numérico fuera de escala ("OverallQual": 99, superando el límite máximo de 10), mientras las 10 variables restantes conservan sus valores válidos de control.

Anexo 5. Bloqueo Preventivo del Servidor (HTTP 422 Unprocessable Content) y Detalle de Validación Pydantic

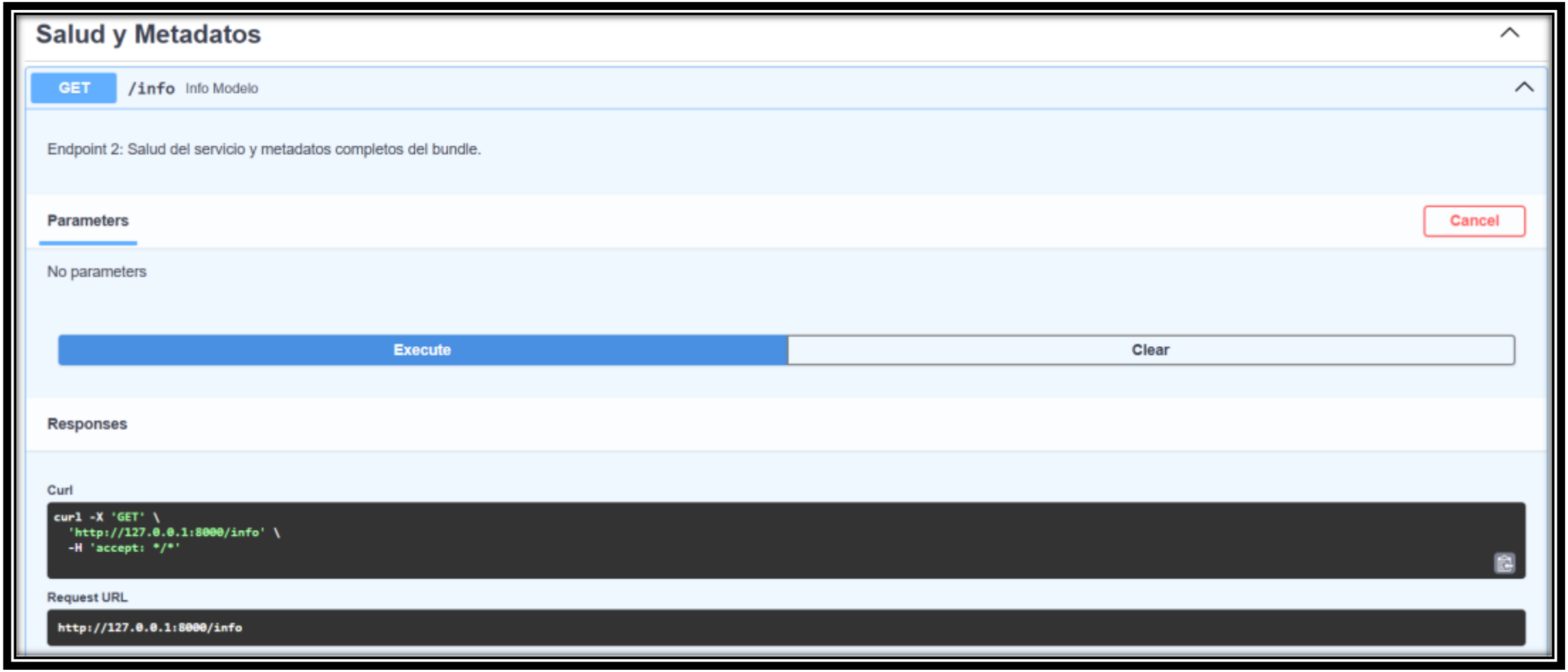


Interpretación:

Se evidencia la respuesta inmediata del servidor **uvicorn** con el código de estado **HTTP 422 (Error: Unprocessable Content)**, demostrando que la capa de validación bloqueó la petición antes de que llegara a distorsionar el modelo predictivo. El cuerpo de respuesta (Response body) detalla con exactitud los dos motivos del rechazo detectados simultáneamente:

- **Violación de rango numérico (less_than_equal):** Identifica en la ubicación ["body", "OverallQual"] el valor ingresado ("input": 99) e informa que el dato debe ser menor o igual a 10 ("le": 10).
- **Violación de categoría permitida (literal_error):** Identifica en la ubicación ["body", "CentralAir"] el texto inválido ("input": "TAL_VEZ") y especifica que el sistema únicamente acepta los valores 'Y' o 'N'.

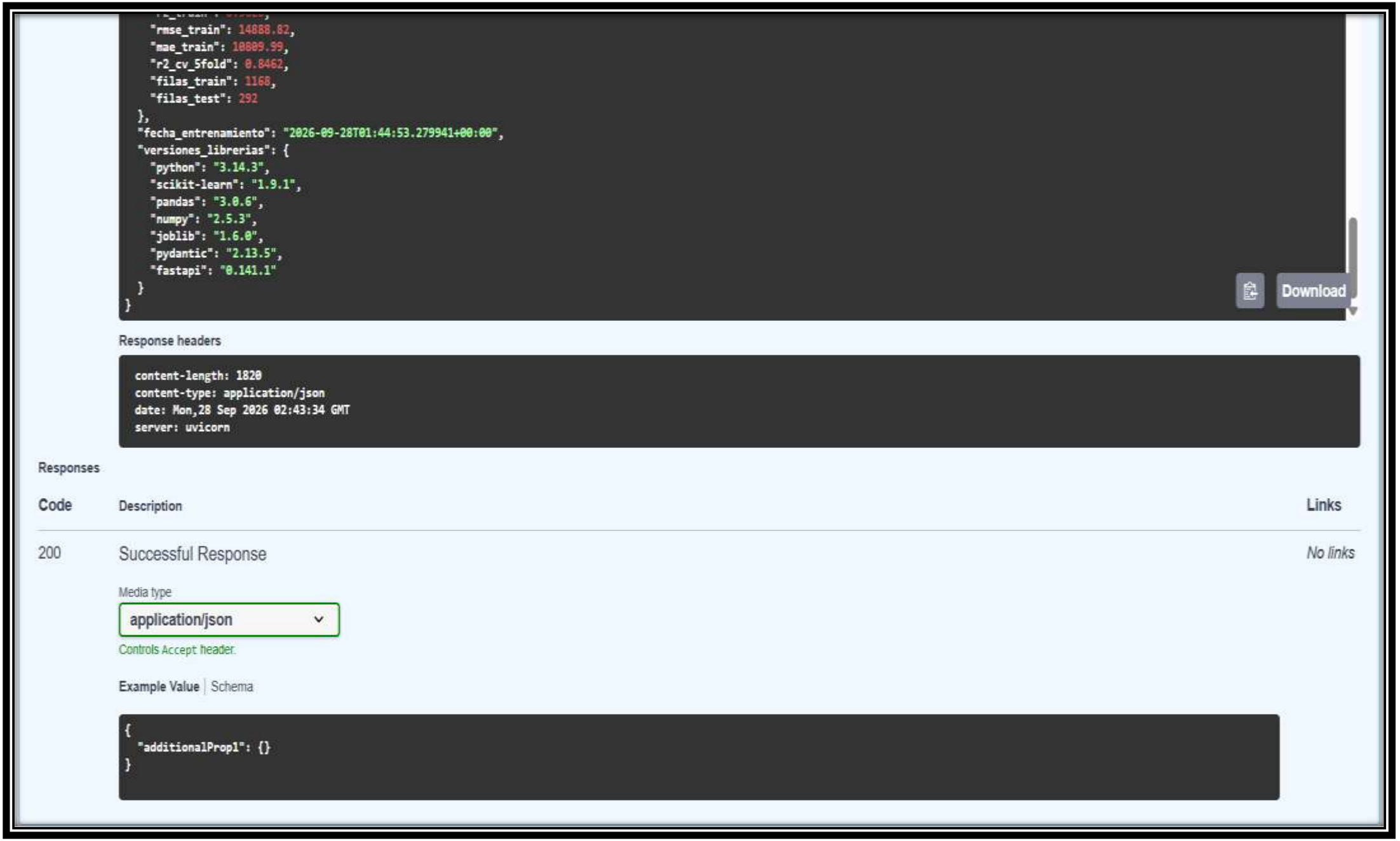
Anexo 6. Ejecución de Consulta de Diagnóstico y Auditoría en el Endpoint GET /info (opcional)



Interpretación:

Se muestra la activación del segundo endpoint del servicio (**GET /info — Salud y Metadatos**), el cual permite auditar en tiempo real el estado interno del contenedor y consultar la ficha técnica del modelo sin necesidad de enviar parámetros de entrada (No parameters). En la sección inferior se observa la instrucción estándar generada (curl -X 'GET') apuntando directamente a la dirección de consulta **http://127.0.0.1:8000/info** para extraer los metadatos almacenados en el archivo serializado .joblib.

Anexo 7. Respuesta de Metadatos, Métricas de Entrenamiento y Versiones del Entorno (HTTP 200 OK)



Interpretación:

Se evidencia la respuesta exitosa del servidor **uvicorn** (content-length: 1820, application/json) exponiendo la trazabilidad completa guardada dentro del bundle en producción:

- **Indicadores de entrenamiento y partición de datos:** Reporta las métricas del conjunto de entrenamiento (rmse_train: 14888.82, mae_train: 10809.99), la validación cruzada de 5 pliegues (r2_cv_5fold: 0.8462) y el volumen exacto de registros utilizados (1,168 filas de entrenamiento y 292 filas de prueba).
- **Sello temporal de compilación:** Registra la fecha y hora exacta en que fue entrenado el modelo en formato universal UTC (2026-09-28T01:44:53.279941+00:00).
- **Inventario de versiones de software (versiones_librerias):** Certifica la compatibilidad técnica del entorno de producción listando las versiones exactas con las que opera el contenedor (python: 3.14.3, scikit-learn: 1.9.1, pandas: 3.0.6, numpy: 2.5.3, joblib: 1.6.0, pydantic: 2.13.5 y fastapi: 0.141.1).